Appendix I
Cooja Trace Generation and Parsing

Sohail Abbas, Muhammad Kazim, Muhammad Fayaz, and Harun Pirim

1 Purpose of This Appendix

This appendix expands the simulation and parsing protocol used in the paper. The goal is to make clear what was obtained from Cooja/Contiki-NG, what was computed after parsing, and what was deliberately excluded from the model inputs. This matters because decreased-rank attack (DRA) detection can easily become inflated if an implementation-level attack log is accidentally used as a feature.

2 Cooja/Contiki-NG Scenario

The simulated network contains one RPL root and 49 non-root motes. The malicious-node ratios are 10%, 20%, and 30%, corresponding to 5, 10, and 15 DRA motes. For each ratio, five independent seeds are used. The seeds change attacker placement and network realization, so the held-out test seed evaluates generalization to an unseen topology and attacker placement rather than memorization of one trace.

Table 1: Trace-generation settings.

Setting	Value
Simulator	Cooja/Contiki-NG
Routing protocol	RPL
Network size	50 motes
Root	1 root, indexed as node 0
Attack type	Decreased-rank attack in DIO advertisements
Attack ratios	10%, 20%, 30%
Seeds per ratio	5
Simulation time	600 s
Warm-up removed	first 60 s
Observation window	60 s
Total logs	15
Graph snapshots	135
Node-level samples	6750

3 Trace-Driven Workflow

Figure 1 shows the complete trace-driven pipeline. The Cooja/Contiki-NG simulations generate serial logs under multiple DRA ratios and seeds. The parser converts these logs into fixed-width observation windows, extracts observable node and packet fields, and keeps DRA audit events separate. The flat-feature branch receives only the node-feature matrix, whereas graph models additionally receive the relation-specific adjacency matrices. This separation is important because it makes the comparison about representation structure rather than about access to different data sources.

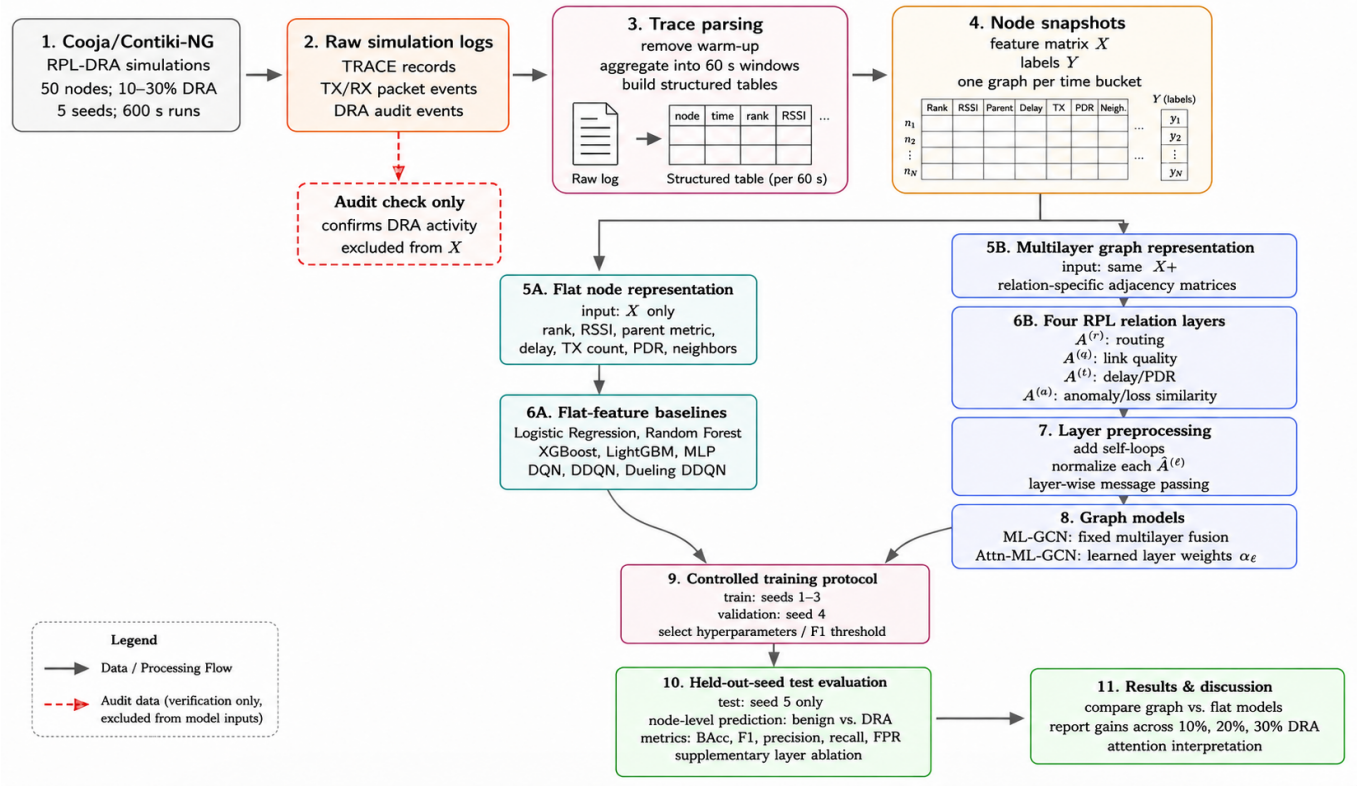

Figure 1: Trace-driven experimental workflow from Cooja/Contiki-NG simulation logs to flat-feature and multilayer graph model evaluation.

4 DRA Mechanism

Let R_i denote the actual RPL rank of node v_i and R_i^{adv} denote the rank that the node advertises in DIO control messages. Under benign operation, the advertised rank is equal to the actual rank:

$$R_i^{adv} = R_i. \tag{1}$$

For a malicious node $v_i \in \mathcal{M}$, the DRA implementation advertises an artificially improved rank:

$$R_i^{adv} = \max(R_{\min}, R_i - \delta_i), \qquad \delta_i > 0. \tag{2}$$

Here $R_{\min}$ prevents the advertised rank from becoming invalid, and δ_i is the malicious rank decrement. The attack is therefore not a direct packet-content attack. It manipulates the control-plane signal used by neighboring motes for parent selection.

A neighboring node v_j chooses its preferred parent from candidate neighbors $\mathcal{N}_j$. In simplified form, parent selection can be represented as

$$p(j) = \arg\min_{k \in \mathcal{N}_j} \Psi(R_k^{adv}, Q_{jk}), \tag{3}$$

where Q_{jk} summarizes link or parent cost. A DRA node can attract traffic because it reduces the first argument of the parent-selection objective without necessarily improving the true route quality.

5 Log Families Produced by the Simulation

The serial output is parsed into four separated artifact families:

1. **Scenario metadata**: attack ratio, seed, malicious-node set, and topology metadata.

2. **Periodic node traces**: node id, rank, DAG rank, preferred parent, parent metric, RSSI, neighbor count, role, and scenario label.

3. **Packet events**: transmission count, successful receptions, delivery delay, and packet-delivery ratio.

4. **DRA audit records**: implementation-level messages showing that a malicious node advertised a manipulated rank.

6 Feature Construction

For every node v_i in an observation window, the parser constructs the observable feature vector

$$x_i = [\tilde{i}, \tilde{R}_i, \tilde{G}_i, \tilde{S}_i, \tilde{M}_i, \tilde{N}_i, \tilde{D}_i, \tilde{T}_i, \tilde{P}_i], \tag{4}$$

where $\tilde{i}$ is the normalized node identifier, $\tilde{R}_i$ is rank, $\tilde{G}_i$ is DAG rank, $\tilde{S}_i$ is RSSI, $\tilde{M}_i$ is parent metric, $\tilde{N}_i$ is neighbor count, $\tilde{D}_i$ is mean delay, $\tilde{T}_i$ is the transmission count, and $\tilde{P}_i$ is PDR. These fields are observable from node and packet traces. The advertised-rank decrement, attack hook state, and internal audit flags are excluded.

7 Leakage Control

The DRA audit record is useful for validating that the scenario executed correctly, but it is not a legitimate detector input. If an audit field were included, the model would learn the simulator's attack marker rather than the network behavior caused by the attack. The pipeline therefore applies the following rule:

$$X = g(\text{node traces}, \text{packet events}), \qquad X \cap g(\text{DRA audit}) = \emptyset. \tag{5}$$

This rule is also applied to graph construction. Routing, link-quality, temporal, and trust/anomaly layers are built from observable routing and packet behavior, not from direct attack instrumentation.

8 Why the Held-Out-Seed Split Matters

A random split over node snapshots can put highly similar windows from the same topology into both training and testing. The paper avoids this by splitting at the Cooja-seed level: seeds 1–3 are used for training, seed 4 for validation, and seed 5 for final testing. The final test therefore contains unseen attacker placements and network realizations, which is a stricter check for publication than random sample splitting.

9 Repository Artifacts

The relevant repository folders are:

- `cooja/`: Contiki-NG/Cooja files and DRA simulation material.
- `scripts/parse_cooja_logs.py`: parser used to generate structured trace tables.
- `data/cooja_clean_5seed/`: parsed trace-derived benchmark data.
- `results/`: exported metrics and figures generated from the parsed snapshots.